



**COMSATS University Islamabad,
Sahiwal Campus.**

Imaan Link

A Project Presented To

**COMSATS University Islamabad,
Sahiwal Campus**

In partial fulfillment

of the requirement for the degree of

Bachelor of Science in Computer Science (2021-2025)

By

Muhammad Hunzla

CIIT/FA21-BCS-107/SWL

Wajid Malik

CIIT/FA21-BCS-146/SWL

Muhammad Ziyad

CIIT/FA21-BCS-201/SWL



**COMSATS University Islamabad,
Sahiwal Campus.**

Imaan Link

By

Muhammad Hunzla

CIIT/FA21-BCS-107/SWL

Wajid Malik

CIIT/FA21-BCS-146/SWL

Muhammad Ziyad

CIIT/FA21-BCS-201/SWL

Supervisor

Mr. Muhammad Jamil

Bachelor of Science in Computer Science (2021-2025)

The candidate confirms that the work submitted is their own and appropriate credit has been given where reference has been made to the work of other.

Declaration

We hereby declare that this software, neither whole nor as a part has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other. We will stand by the consequences. No Portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Muhammad Hunzla

Wajid Malik

Muhammad Ziyad

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS(CS) “**Imaan Link**” was developed by **Muhammad Hunzla**(CIIT/FA21-BCS-107/SWL) , **Wajid Malik** (CIIT/FA21-BCS-146/SWL) and **Muhammad Ziyad** (CIIT/FA21-BCS-201/SWL) under the supervision of “**Mr. Muhammad Jamil**” and co-supervisor “**Mr. Arslan Sarwar**” and that in their opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in Computer Science.

Supervisor

Mr. Muhammad Jamil

Lecturer

Department of Computer Science

COMSATS University Islamabad, Sahiwal Campus

Head of Department

Dr. Zafar Iqbal Roy

Assistant Professor

Department of Computer Science

COMSATS University Islamabad, Sahiwal Campus

External Examiner

Dr Sana Yasin

Lecturer

Department of Computer Science

University of Okara

Executive Summary

Imaan Link is a modern digital platform created to serve the evolving religious, educational, and community needs of the global Islamic population. Recognizing the challenges faced by individuals and institutions in hiring qualified religious scholars, accessing authentic Islamic resources, and managing religious events, this project provides a centralized and user-friendly solution through both mobile and web applications.

The platform features multiple integrated modules, including Scholar Hiring, Profile Management, Event Management, Tasbih Counter, Community Discussions, and a Resource Library (Quran and Duas). Each module is designed to support accessibility, trust, and seamless interaction among users, scholars, and mosque administrators.

Developed using Flutter for a cross-platform frontend and Firebase for real-time backend services, Imaan Link is scalable, secure, and optimized for a smooth user experience. The project follows an Agile methodology, enabling iterative development, stakeholder feedback integration, and timely module delivery.

The key impact of Imaan Link lies in its ability to connect users with verified scholars, simplify religious engagement through technology, and promote learning and collaboration within the Islamic community. By leveraging cloud infrastructure (real time chat), and real-time notifications, Imaan Link transforms how Muslims around the world access and engage with religious services in the digital age.

Acknowledgement

All praise is to Almighty Allah who bestowed upon us a minute portion of His boundless knowledge by virtue of which we were able to accomplish this challenging task.

We are greatly indebted to our project supervisor “Mr. Muhammad Jamil” and our Co-Supervisor “Mr. Arslan Sarwar”. Without their personal supervision, advice and valuable guidance, completion of this project would have been doubtful. We are deeply indebted to them for their encouragement and continual help during this work.

And we are also thankful to our parents and family who have been a constant source of encouragement for us and brought us the values of honesty & hard work.

Muhammad Hunzla

Wajid Malik

Muhammad Ziyad

Abbreviations

Abbreviation	Meanings
SRS	Software Require Specification
SDD	Software Design Document
FCM	Firebase Cloud Messaging
NS	Notification System
FR	Functional Requirement

Table of Contents

1	Introduction.....	1
1.1	Brief.....	1
1.2	Relevance to Course Modules	1
1.2.1	Mobile App Development (CSC303).....	1
1.2.2	Human-Computer Interaction (CSC356)	2
1.2.3	Operating Systems (CSC322)	2
1.2.4	Database Systems I (CSC371)	2
1.3	Project Background	2
1.4	Literature Review	2
1.5	Analysis from Literature Review	3
1.6	Methodology and Software Lifecycle for This Project	3
1.7	Key Phases of the Methodology	4
1.7.1	Requirement Analysis	4
1.7.2	Implementation	4
1.7.3	Testing.....	4
1.7.4	Deployment & Maintenance.	4
1.8	Rationale behind Selected Methodology	5
2	Problem Definition	6
2.1	Problem Statement.....	6
2.2	Deliverables and Development Requirements.....	6
2.2.1	Mobile Application (Android)	6
2.2.2	Admin Panel (Separate App base)	7
2.2.3	Fire Base real time Authentication.....	7
2.2.4	Documentation	7
2.2.5	Testing & Quality Assurance	7

2.3	Development Requirements	7
2.3.1	Technology.....	7
2.3.2	Skills Required.....	7
2.3.3	Timeline	7
2.3.4	App Deployment	7
2.4	Current System VS Proposed System	8
2.5	Requirement Analysis.....	8
2.6	Detailed Use Case.....	8
2.6.1	Use case.....	9
2.7	Use Case Description.....	9
2.8	Functional Requirements	18
2.9	Non-Functional Requirements.....	19
2.10	Admin Panel	19
2.11	Key Modules	20
3	Design and Architecture.....	21
3.1	System Architecture.....	22
3.2	Process Flow.....	23
3.3	Process Flow/Representation.....	24
3.3.1	Flow Description.....	24
3.4	Sequence Diagram.....	25
3.5	Class Diagram.....	26
		26
4	Implementation	27
4.1	External APIs	27
4.2	User Interface	27
4.2.1	Dashboard and Job Tab.....	28
4.2.2	Profile and More Options.....	29
4.2.3	Admin Dashboard and management	30

4.2.4	Verification request and Report History	31
		31
4.2.5	User management and report content view	32
4.2.6	Verification Requests and New admin Creation	33
5	Testing and Evaluation	34
5.1	Manual Testing	34
5.2	System Testing	34
5.3	Unit Testing	35
5.4	Functional Testing	37
5.5	Integration Testing.....	38
5.6	Automated Testing.....	39
6	Conclusion and Future Work.....	40
6.1	Conclusion.....	40
6.2	Future Work.....	41
7	References.....	43

List of Figures

Figure 1:Use Case	9
Figure 2:Architecture Diagram	22
Figure 3:Process Flow.....	23
Figure 4:Sequence Diagram.....	25
Figure 5:Class Diagram	26
Figure 6:Job Tab	28
Figure 7:Dashboard.....	28
Figure 8: User Profile.....	29
Figure 9: More Options.....	29
Figure 10:Admin Management	30
Figure 11:Admin Dashboard.....	30
Figure 12:Report History	31
Figure 13:Verification.....	31
Figure 14:Post Review	32
Figure 15:User Management.....	32
Figure 17:New admin Creation.....	33
Figure 16:Verification Request.....	33

List of Tables

Table 1.6.1: This Table Shows that the features that app is delivering.....	4
Table 2:Login.....	9
Table 3:Register	10
Table 4:Create Update Profile	11
Table 5:View Posts	12
Table 6:Send connection invite and messages(Future).....	12
Table 7:Add/Delete and Update Posts	13
Table 8:Add job/Post Jobs	14
Table 9:Find jobs	15
Table 10:Create and join Evenet/Community.....	16
Table 11:Question Answer Section.....	17
Table 12.10 shows that the feature that alongside with their roles that are added in the admin panel/Dashboard	20
Table 13: Describes the API'S and their description.....	27
Table 14:The below table described the features and expected outcomes of the testing.....	34
Table 15:The below table described the Unit testing using inputs to reach the expected results	35
Table 16:The below table shows that the integration testing and its expected results.....	38
Table 17:Describes the Tools and their uses in the app	39

Chapter 1

Introduction

1 Introduction

In today's digitally connected world, the way people access religious services is undergoing a significant transformation. However, despite this shift, many Islamic services still rely on manual, offline processes. Whether it's hiring a scholar, attending a mosque event, or engaging with religious communities, users often face fragmented experiences spread across multiple apps or offline networks.

Imaan Link addresses these challenges by creating a centralized Islamic digital platform. It provides users access to verified scholars, a community engagement system, spiritual tracking tools (like Tasbeeh), and an administrative ecosystem to ensure authenticity and moderation.

With the recent introduction of a dedicated Admin App, Imaan Link has expanded its capability beyond a traditional user app. Admins can now manage reports, verify certificates, and moderate content through an entirely separate application, enhancing scalability, security, and operational control.

The goal of this project is to bring Islamic services in line with 21st-century digital standards while maintaining religious authenticity, trust, and simplicity for all users.

1.1 Brief

Imaan Link is a cross-platform digital solution comprising two core applications:

- **Main App:**
 - Scholar hiring system
 - Enhanced Tasbeeh counter with Dikr suggestions and goal setting
 - Community module with post types (text, image, video)
 - Event and feedback systems
 - Verified badge on scholar profiles
- **Admin App:**
 - Full admin dashboard with analytics
 - Report management (3-warning system)
 - Super Admin panel to create/update/delete admins
 - Scholar verification with document review
 - Report resolution tracking

Both apps are developed using Flutter (mobile/web) and backed by Firebase for Authentication, Realtime Database, Cloud Messaging, and Storage.

1.2 Relevance to Course Modules

This project directly aligns with several key course modules:

1.2.1 Mobile App Development (CSC303)

From studying web technologies, we learned the diverse aspects of website development, ranging from basic static pages to complex web applications. This understanding was vital in crafting my project's online platform, encompassing various web-based functionalities.

1.2.2 Human-Computer Interaction (CSC356)

Aligning with this course, my project prioritizes user-friendly interfaces across system modules. By integrating principles of human-computer interaction, I aim to create an intuitive system. This project application allows me to implement and assess user interface design, refining my skills in usability and user experience testing.

1.2.3 Operating Systems (CSC322)

Understanding operating systems was crucial for my project, as it forms the backbone of computer applications. We comprehended how applications interact with an operating system through an application program interface (API), contributing to a better grasp of system-level interactions.

1.2.4 Database Systems I (CSC371)

My project serves as a practical application of database management concepts learned in this course. It enables hands-on experience in designing and implementing databases, allowing me to translate theoretical database knowledge into real-world applications, emphasizing data storage and manipulation skills.

1.3 Project Background

In most Islamic communities, finding qualified religious figures, attending events, or even accessing authentic learning materials is often done through word-of-mouth or fragmented mobile apps. This leads to inefficiency, misinformation, and lack of transparency.

Imaan Link was conceptualized to provide a centralized, digitally accessible, and verified ecosystem where all these services are unified. By allowing users to hire scholars, track events, rate services, and interact with communities, the platform contributes to strengthening Islamic practices through convenience and technological empowerment. Islamic services scholar access, learning events, community discussions are essential in the religious lives of Muslims. Traditionally, these are handled through word-of-mouth, mosque bulletin boards, or unverified social platforms.

This approach leads to:

- Lack of scholar verification
- Limited accessibility to events/resources
- No structured way to engage online as a religious community

Imaan Link solves this with an integrated system that offers trust, visibility, and control to users and administrators alike. (360, n.d.)

1.4 Literature Review

Several platforms exist for limited Islamic services:

- **Muslim Pro:** Known for prayer times and Quranic resources, lacks scholar interaction or hiring.
- **Islam 360:** Excellent learning resource, lacks hiring or community features.
- **Quran Majeed:** Offers recitation and Tafsir, but has no event or community modules.

Despite these efforts, no solution yet combines service hiring, education, event management, and feedback

in one app. Research also shows users desire secure, easy-to-navigate platforms with real-time updates and Islamic authenticity. Imaan Link is designed to fulfill these exact requirements.

Gaps identified:

- No community moderation
- No hiring workflows
- No admin-controlled ecosystem

Imaan Link fills these gaps with real-time scholar services, community moderation, and admin operations.

1.5 Analysis from Literature Review

The literature indicates a fragmented experience for Muslim users seeking digital services. Users often download multiple apps for prayers, Quran, local events, and scholar connections. Imaan Link not only addresses these gaps by offering an all-in-one solution, but it also builds trust through verified scholars, engagement through discussion forums, and ease through a unified dashboard.

From the analysis, we concluded that Islamic apps are either:

- Resource-focused (Quran, Hadith)
- Utility-focused (Qiblah, prayer time)
- Or unmoderated forums (groups, social media)

Imaan Link combines all three with:

- Moderated community
- Verified scholar services
- Interactive tools like Tasbeeh and feedback

This makes it a **hybrid platform**—educational, social, and service-oriented.
a more objective and reliable data source.

1.6 Methodology and Software Lifecycle for This Project

The Agile methodology was selected, supported by Iterative Development. Agile facilitates the flexibility needed to evolve the app based on real-time feedback from scholars, mosque administrators, and end-users.

The lifecycle followed these major steps:

- Requirements gathering through user interviews and surveys
- Sprint planning and backlog prioritization
- Modular development in short cycles (sprints)
- Unit and integration testing
- Final review and feature enhancement based on stakeholder input

Table 1.6.1: This Table Shows that the features that app is delivering.

Sprint	Features Delivered
1	User Auth, Profile Module
2	Scholar Profiles, Hiring Requests
3	Community Module (text/image/video)
4	Tasbeeh counter goals + animations
5	Admin Dashboard (v1)
6	Admin: Report handling, certificate verification
7	Final refinements, verified badge, post moderation

1.7 Key Phases of the Methodology

The key phases show describes the project requirement, system design, implementation, testing and deployment.

1.7.1 Requirement Analysis

Requirements were gathered from the proposal, academic guidelines, and user expectations. The focus was on simplicity, online capability, and ease of hiring scholars without suffering from physical exhaustion.

1.7.1.1 System design

A modular architecture was designed. The frontend was built using Flutter (based UI), while the backend model was developed using Dart and Firebase

1.7.2 Implementation

The application was implemented in multiple builds. The first version handled UI and navigation, followed by data input validation.

1.7.3 Testing.

Each increment was tested individually using real-world users testing.

1.7.4 Deployment & Maintenance.

The app was packaged as an APK and made available through an update link. Maintenance includes retraining the community posts.

1.8 Rationale behind Selected Methodology

We avoided traditional Waterfall models due to their rigidity. Agile allowed us to:

- Adjust features based on scholar feedback
- Test each module independently
- Engage users from early development phases

Refactor components without affecting the whole system.

Agile was chosen because:

- Requirements evolved during implementation
- Admin app was added mid-project
- UI/UX improvements were made after usability testing
- It allows parallel module development

This flexibility helped scale the system smoothly.

Chapter 2

Problem Definition

2 Problem Definition

There is no centralized, digital solution for accessing verified religious scholars, Islamic resources, and community engagement tools. People rely on fragmented apps or offline methods that lack security, traceability, and convenience.

2.1 Problem Statement

In the digital age, access to verified and authentic Islamic services remains a significant challenge. While many areas of life have transitioned to online platforms, religious service delivery—especially in the Islamic world—continues to rely heavily on manual processes, unverified sources, and fragmented solutions. People seeking religious scholars for mosque duties or educational purposes often depend on local contacts or word of mouth, with no system in place to validate the qualifications of those they are engaging. This results in limited access, inefficiency, and in some cases, mistrust due to the lack of standardized verification.

Additionally, community engagement among Muslims online is either highly informal or managed through unmoderated platforms like WhatsApp groups or Facebook pages. These lack structured environments, leading to the spread of misinformation, inappropriate content, and little to no accountability. There is also no way to escalate concerns, report problematic content, or manage scholarly credentials in these spaces.

The lack of a central platform that integrates scholar hiring, community interaction, post moderation, and administrative oversight reflects a critical gap. Furthermore, traditional tasbeeh applications offer minimal user interaction and lack features that could help Muslims track their spiritual goals in a more meaningful way. In the absence of an integrated ecosystem for all of these services, users are left juggling multiple platforms and uncertain sources.

Imaan Link is designed to directly address these gaps by offering a comprehensive, trusted, and secure platform that not only allows verified scholar engagement but also facilitates moderated community discussions, personalized spiritual tracking, and administrative control—all under one digital umbrella.

2.2 Deliverables and Development Requirements

Deliverables and development requirements.

Deliverables:

2.2.1 Mobile Application (Android)

- Mobile application (Android/iOS via Flutter)
- Web portal for administrators and scholars
- Functional modules: Hiring, Profile, Events, Education, Community, Tasbih
- Role-based user access

2.2.2 Admin Panel (Separate App base)

- Manage user, Admin, Posts, Reports
- Add, update, or delete Admins
- History of Solved Problem

2.2.3 Fire Base real time Authentication

- Real time authentication of users for login
- Authenticated Scholars Option
- Verified badge for verified users next to profile

2.2.4 Documentation

- **User Manual** with instructions on how to use the app and how to get verified

2.2.5 Testing & Quality Assurance

- Thorough testing of the app on multiple devices (unit tests, UI tests).
- **User Acceptance Testing (UAT)** to ensure the app meets user needs and works as expected.
- **Performance Testing** to ensure smooth operation on various Android devices.

2.3 Development Requirements

2.3.1 Technology

- **Frontend:** Android development using **Flutter** and **Dart** for UI and Backend
- **Real time Chat:** User can chat with each other and chat is stored in firebase
- **Backend:** Dart and firebase for backend management (for user data, updates, etc.).
- **Push Notifications:** **Firebase Cloud Messaging (FCM)** for sending Notification

2.3.2 Skills Required

- **Dart:** Expertise in Android development, especially for UI design and data management.
- **Fire base:** Familiarity with firebase for easy user handling
- **Firebase Integration:** Knowledge of Firebase for user authentication, notifications, and analytics.
- **UI/UX Design:** Understanding of material design principles to create an intuitive and user-friendly app.

2.3.3 Timeline

- **Phase 1:** Project Setup & Initial Planning (1–2 weeks)
- **Phase 2:** Resource Gathering (3–4 weeks)
- **Phase 3:** App Development (UI/UX, Notifications) (4–6 weeks)
- **Phase 4:** Testing, Debugging, & User Feedback (2–3 weeks)
- **Phase 5:** Final Adjustments, Deployment, and Launch (1–2 weeks)

2.3.4 App Deployment

- Deploy the app to the **Google Play Store** for public access or distribute it directly via APK for private distribution.
- Post-launch monitoring for user feedback and bug fixes.

2.4 Current System VS Proposed System

Traditional Approach:

- No way to verify scholar credibility
- Mosque events are not published online
- No centralized reporting or moderation
- Tasbeeh apps lack customization and goal

Imaan Link System:

- Verified scholar hiring
- Admin-managed content
- Goal-based tasbeeh with feedback
- Modular system with future expandability

The proposed system, Imaan Link, overcomes these limitations by introducing a comprehensive platform with user verification, structured community interaction, feedback and rating mechanisms, and a completely separate admin control panel. The admin app allows for real-time oversight of flagged content, approval of scholar certifications, and the creation of new admins with role-based authority. Users benefit from a reliable system that promotes engagement, learning, and spiritual discipline in a secure and trustworthy environment.

2.5 Requirement Analysis

The requirement identifying technique for creating a use case diagram for Imaan Link system involves stakeholder analysis, use case identification, use case description, use case relationships, and use case diagram construction.

2.6 Detailed Use Case

Use case descriptions provide a detailed narrative of how the system interacts with various actors to accomplish specific tasks or goals. Each use case outlines the interactions between users, administrators, and the Imaan Link platform, capturing the functional requirements of the system. These descriptions serve as the foundation for understanding system behavior and ensuring all stakeholder needs are addressed during development. Use case descriptions bridge the gap between abstract requirements and tangible system designs. They provide clarity for developers, testers, and stakeholders.

2.6.1 Use case

The use case diagram shows the user and admin interaction in the system.



Figure 1:Use Case

2.7 Use Case Description

The table below indicates a comprehensive use case template filled in with an example drawn from the Imaan Link perspective.

Table 2 describes a login use case. A user of the System logs into the System.

Table 2:Login

Use case id	01
Use case name	Login
Actors	Admin User

Description	A user of the System logs into the System.
Trigger	The user requests to log in.
Pre-condition	The user has an account.
Postcondition	The actor is now logged into the system.
Normal flow	• This use case starts when an actor wants to login into the system. • The system requests that the actor enter e-mail/phone no. and password. • The actor enters the email/phone number and password. • The system validates the entered e-mail/phone no. and password and • login into the system.
Alternative flow	For example, a Valid e-mail/Phone number or an Invalid Password If in the basic flow the system cannot find the valid e-mail or the password is invalid, an error message is displayed.
Exceptions	Incorrect credentials. Request to re-enter the login details.
Business rules	The system must be connected to the network.

Table 3 describes the use case of the registration process. A user of the System creates an account.

Table 3: Register

Use case id	02
Use case name	Get Registration
Actors	• user • - Admin
Description	A user of the System creates an account.
Trigger	Select the register link
Pre-condition	When the user doesn't have an account to log in to the system.

Postcondition	The user's details are stored in the database.
Normal flow	• This use case starts when the user is not logged in to the system and goes to the login page. • The system prompts the user to send an e-mail and password or register a new account. • The user selects the registration option. • The system displays the signup page that allows users to fill in their username/email address, first and last name, and password. • The user enters their information. • The system verifies information and creates an account.
Alternative flow	• Cancel Registration • The users select the cancel option. • The system returns the user to the home page without the user being logged in and any information entered has been erased. • Invalid Information Entered • Users can enter information and submit it. • The system displays a message to correct invalid information. The user re-enters the information.
Exceptions	If an existing user exists, then a "User already exists" message is displayed. If information is missing "Insufficient Information" message is displayed.

Table 4 describes the profile of users Users create and add a new profile with personal, professional, and educational details.

Table 4: Create Update Profile

Use case id	03
Use case name	Create Update Profile
Actors	• User
Description	Users create and add a new profile with personal, professional, and educational details.
Trigger	Users decide to create a new profile.
Pre-condition	Users are logged in; the system is operational.
Postcondition	The profile is successfully created and stored in the system.

Normal flow	• Users log in. • Navigate to 'Create Profile'.
	• Enter personal, professional, and educational details. • Review and submit profile. • System saves profile and confirms creation.
Alternative flow	Prompts for missing information; allows profile updates.
Exceptions	System unavailability; validation errors; session timeout.

Table 5 describes the posts where users can view posts.

Table 5:View Posts

Use case id	03
Use case name	View posts
Actors	• User
Description	Users can view all the posts and click the Post of his/her own choice.
Trigger	Users want to request to see the post details.
Pre-condition	The actors log in to the system
Postcondition	The user sees the information related to a particular post and interact with question-answer sessions and can create their own as well.
Normal flow	The users log in to the system. View the posts or feed
Alternative flow	The user did not register his /herself and did not view the available posts and question answers details.
Exceptions	The error occurs in the application. Server is down

Table 6 describe the connectivity section.

Table 6:Send connection invite and messages(Future)

Use case id	04
-------------	----

Use case name	Send connection invite and messages
Actors	User-to-user
Description	Users send messages and connect with other users within the job hosting application to network, discuss job opportunities, or share information.
Trigger	Users decide to initiate a conversation or respond to a message from another user.
Precondition	Users are logged in; the system's messaging feature is operational.
Postcondition	Messages are successfully sent, received, and stored in the system, and connections are established between users.
Normal flow	• Users log in. • Navigate to the 'Messages' or 'Connections' section. • Search for or select the user they want to connect with or message. • Compose and send a message. • The recipient receives the message and can respond. • System stores the conversation history.
Alternative flow	Users search for a specific user, initiate a connection request, and wait for acceptance before messaging.
Exceptions	System unavailability; messages fail to send. Validation errors; incorrect or inappropriate message content. The user blocks another user; the message cannot be sent.

Table 7 describes the delete and update functionality of the system.

Table 7: Add/Delete and Update Posts

Use case id	05
Use case name	Add/Delete and Update Posts
Actors	□ user

Description	Users add, delete, and update posts within the application to share job opportunities, articles, updates, or other relevant information
Trigger	Users decide to create, modify, or remove a post.
Pre-condition	Users are logged in; the post-management feature is operational.
Postcondition	Posts are successfully created, updated, or deleted, and changes are reflected in the system.
Normal flow	1. Users log in. 2. Navigate to the 'Posts' section. 3. To add a post, users select the option to create a new post, enter the content, and submit it. 4. To update a post, users select an existing post, edit the content, and save the changes. 5. To delete a post, users select an existing post and choose the delete option. 6. System processes the request and updates the post status (added, updated, or deleted).
Alternative flow	• Users can preview a post before submitting or updating it. • Users can revert to a previous version of an updated post if necessary.
Exceptions	• System unavailability; adding, updating, or deleting posts fails. • Validation errors; missing or incorrect post content. • Insufficient permissions; users are not allowed to modify or delete certain posts.
Business rules	• The system must be connected to the network. • Search results should be relevant and up-to-date. • The system should provide filtering and sorting options to refine job search results.

Table 8 describes the listing of jobs

Table 8:Add job/Post Jobs

Use case id	06
Use case name	Add Jobs
Actors	user

Description	Employers or authorized users add new job listings and update existing job listings within the job hosting application to advertise open positions.
Trigger	Employers or authorized users decide to create a new job listing or update an existing one.
Precondition	Users are logged in with the necessary permissions; the job management feature is operational.
Postcondition	Job listings are successfully created or updated, and changes are reflected in the system.
Normal flow	• Users log in. • Navigate to the 'Jobs' section. • To add a job, users select the option to create a new job listing, enter job details (title, description, requirements, location, etc.), and submit it. • To update a job, users select an existing job listing, edit the details, and save the changes. • System processes the request and updates the job listing status (added or updated).
Alternative flow	• Users can preview a job listing before submitting or updating it. • Users can duplicate an existing job listing to create a similar new job.
Exceptions	System unavailability; adding or updating job listings fails. Validation errors; missing or incorrect job details. Insufficient permissions; users are not allowed to modify certain job listings.

Table 9 describes the job users search for and find job listings within the job hosting application based on various criteria such as job title, location, industry, and more.

Table 9:Find jobs

Use case id	07
Use case name	Find Jobs

Actors	Users
Description	users search for and find job listings within the job hosting application based on various criteria such as job title, location, industry, and more.
Trigger	Users decide to look for job opportunities
Pre-condition	Users are logged in; the job search feature is operational.
Postcondition	Users successfully find relevant job listings based on their search criteria.
Normal flow	• Users log in. • Navigate to the 'Find Jobs' section. • Enter search criteria (e.g., job title, location, industry, keywords). • Submit the search query. • System processes the search and displays a list of matching job listings. • Users review the search results and can click on job listings for more details.
Alternative flow	• Users can save search criteria for future use. • Users can filter and sort search results by relevance, date posted, company, etc.
Exceptions	• System unavailability; job search fails. • No matching job listings found; the system suggests alternative search criteria or similar jobs. • Validation errors; incorrect or incomplete search criteria

Table 10 describes the description joining communities and hosting communities

Table 10:Create and join Evenet/Community

Use case id	08
Use case name	Create and join events
Actors	• User
Description	Users create new events and join existing events within the job hosting application to participate in job fairs, networking sessions, workshops, and other career-related activities.
Trigger	Users decide to organize a new event or participate in an existing event.
Precondition	Users are logged in; the event management feature is operational.

Postcondition	Events are successfully created or joined, and the system reflects the updated event participation status.
Normal flow	• Users log in. • Navigate to the 'Events' section to create an event • • Select the option to create a new event. • Enter event details (title, description, date, time, location, etc.). • Submit the event for creation. • To join an event browse or search for existing events. • Select an event to join. • Confirm participation. • System processes the requests and updates the event details or the user's participation status.
Alternative flow	Users can edit or cancel the events they have created. Users can withdraw from events they have joined.
Exceptions	System unavailability; event creation or joining fails. Validation errors; missing or incorrect event details. Event capacity reached; users cannot join the event.

Table 11 Users utilize a question-answer section within the application to ask questions, related to their queries, and other users with information can answer their queries and other relevant topics.

Table 11:Question Answer Section

Use case id	09
Use case name	Question Answer Section
Actors	Users
Description	Users utilize a question-answer section within the application to ask questions, related to their queries, and other users with information can answer their queries and other relevant topics.
Trigger	Users decide to ask a question or provide an answer within the question-answer section.
Precondition	Users are logged in; the questionanswer feature is operational.
Postcondition	Questions are successfully posted, and answers are provided within the system.

Normal flow	Users log in. Navigate to the 'Question Answer' section. To ask a question: • Select the option to ask a new question. • Enter the question details. • Submit the question for posting. To provide an answer: • Browse existing questions or search for specific topics. • Select a question to view details. • Enter an answer in the provided text box.
	• Submit the answer. The system processes the requests and updates the question-answer section with the new question or answer.
Alternative flow	Users can browse questions and answers without actively participating. Users can edit or delete their own questions or answers.
Exceptions	System unavailability; posting questions or answers fails. Validation errors; missing or incorrect question or answer details. Inappropriate content detected; questions or answers are flagged for moderation.
Business rules	• The system must be connected to the network. • Questions should be moderated based on community guidelines. • Admin actions should be logged for audit purposes.

2.8 Functional Requirements

The functional requirements include the ability for users to register and authenticate securely, hire scholars based on their availability and expertise, and submit feedback post-engagement. The system also enables users to join communities via a 6-digit code and post various types of content (text, image, video). The admin panel supports certificate verification, report resolution, and creation of additional admins by a super admin.

- **FR1:** User registration and login system with secure authentication.
- **FR2:** Hiring module to request, accept, or reject scholar services.

- **FR3:** Profile module for both hirers and scholars.
- **FR4:** Notification system for hiring updates and event alerts.
- **FR5:** Event module to create, update, and view Islamic events.
- **FR6:** Feedback.
- **FR7:** Tasbih counter
- **FR8:** Resource library (PDF).
- **FR9:** Community discussion

2.9 Non-Functional Requirements

On the non-functional side, the system is expected to be scalable, supporting thousands of users simultaneously with minimal delay. It must be secure, with all user data encrypted and access restricted by clearly defined roles. The interface should be user-friendly and accessible, ensuring ease of use across different age groups and literacy levels. The system must also be reliable and maintain high availability to support real-time interactions without interruption.

- **NFR1:** Availability – 99.5% uptime.
- **NFR2:** Scalability – System should handle 100,000+ concurrent users.
- **NFR3:** Performance – Response time < 2 seconds for core actions.
- **NFR4:** Security – End-to-end encryption, Firebase Authentication.
- **NFR5:** Accessibility – Support for English, (Arabic, Urdu in future).
- **NFR6:** Usability – Clean UI, minimal learning curve.
- **NFR7:** Compatibility – Android 7+, iOS 11+, desktop browsers.

The system design shall allow for potential future development of an iOS version without major architectural changes.

2.10 Admin Panel

The Admin Panel is a critical component of the Imaan Link ecosystem and is implemented as a separate application to enhance security and modularity. It allows administrators to manage the entire platform through a dedicated interface that offers complete visibility and control over user activity, community posts, and scholar verification.

The dashboard provides a summary of key statistics such as the total number of users, active reports, and resolved cases. Reports are visualized using graphs, allowing admins to identify trends in platform usage and community behavior. Each report submitted by users regarding inappropriate or harmful content is reviewed manually. Admins are equipped with a three-strike warning system—on the third violation, the offending post is permanently removed and the user may be restricted depending on the severity of the issue. Reports that do not constitute a violation can be marked as resolved and closed.

A super admin role is also integrated into the system, with the authority to create, delete, or update the roles and permissions of other admins. This ensures scalability and delegation of

duties without compromising platform integrity. Another important feature of the admin panel is certificate verification. When a scholar submits a certificate in the main app for profile verification, the full user profile and supporting document are displayed in the admin interface. The admin can then approve or reject the application. Once approved, the user's profile in the main app is updated with a visible "Verified" badge, indicating credibility to others.

This separation of administrative control from the user experience allows for secure, independent oversight, which is rarely found in religious or community-based applications

2.11 Key Modules

Key modules that are added in the admin app are listed Below:

Table 12.10 shows that the feature that alongside with their roles that are added in the admin panel/Dashboard

Modules	Feature
Dashboard	Admin profile, total reports, user count, graph of reports over time
Report Review	In-depth view of flagged posts, warning system (3-strikes), ability to resolve/delete
Certificate Verification	Admin views user-submitted certificates, validates them, and approves or rejects
Admin Management	Super Admins can create, delete, or modify permissions of admins
History Tracking	Admin can review resolved/dismissed reports, timestamps, and moderators involved

Chapter 3

Design and Architecture

3 Design and Architecture

The architecture of the Imaan Link platform follows a **modular, client-server model** that separates user-facing services from administrative controls. It is composed of two primary applications: the **Main App** for general users and scholars, and a separate **Admin App** for platform managers. Both apps are developed using **Flutter**, ensuring consistent performance across Android, iOS, and web platforms. The backend is fully managed using **Firebase**, which offers a cloud-based, scalable infrastructure including Authentication, Firestore (NoSQL database), Cloud Storage for media, and Firebase Cloud Messaging for real-time notifications. This architecture supports **role-based access control**, where users, scholars, admins, and super admins have different privileges and system visibility, enabling secure, streamlined interactions between modules.

Each module of the system—Hiring, Profile, Tasbeeh, Community, and Admin—is loosely coupled and interacts via Firebase’s real-time database services. This allows for rapid feature integration and independent scalability of each module. For example, when a user requests to hire a scholar, the system stores the request in Firestore and triggers a notification using FCM, all without blocking the user interface. The **Admin App** is designed as a separate environment, allowing for secure and isolated moderation of reports, certificate verification, and administrative tasks. This separation ensures that critical moderation and verification processes are handled outside the main user flow, reducing risk and improving control. The overall design emphasizes **scalability, maintainability, and future extensibility**, making it easy to introduce new features such as live scholar sessions or AI-powered content suggestions.

3.1 System Architecture

The system architecture diagram shows the Architecture of the Imaan Link and how each module is connected to each other and shows the design aspects.

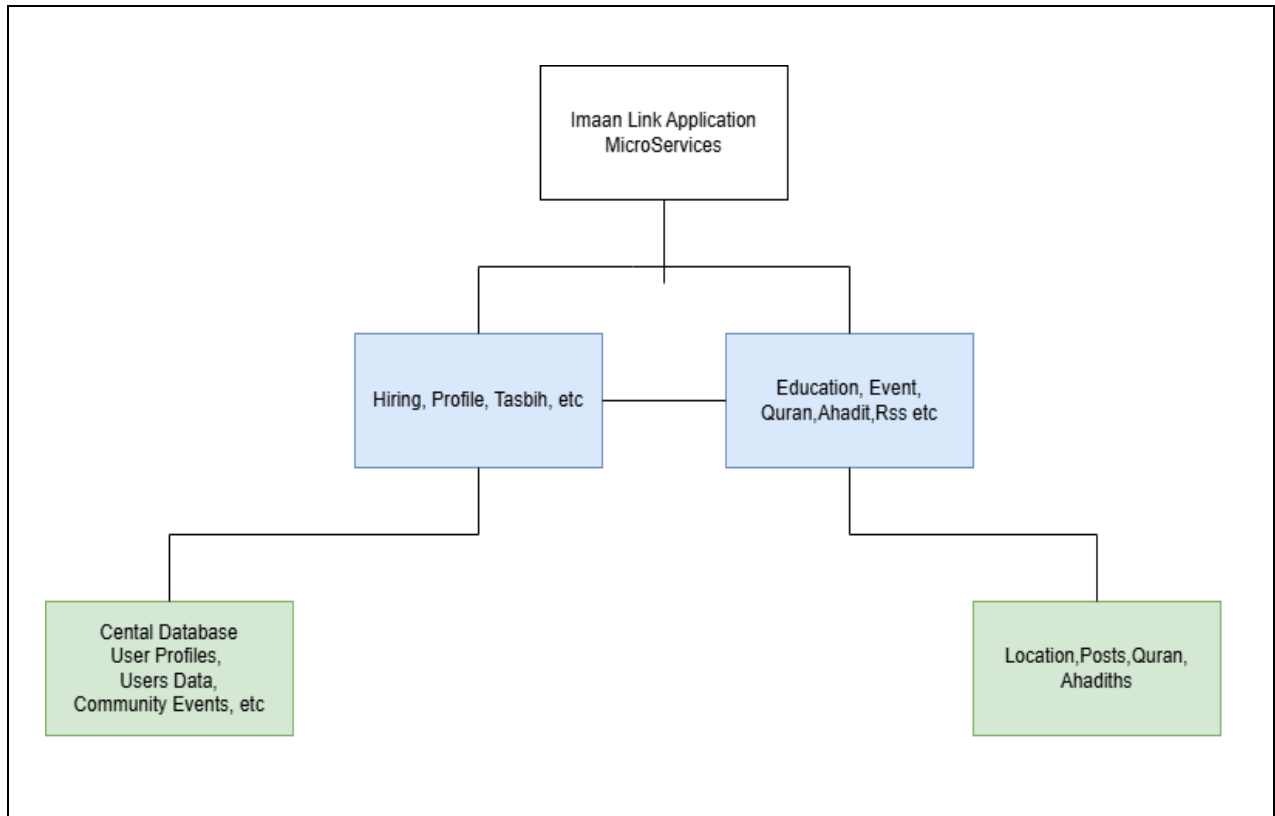


Figure 2:Architecture Diagram

The Imaan Link system is based on a client-server architecture with Platform Based Application The main component is the Android mobile application, which interacts with a Flutter and Dart system, Firebase as Storage, and optionally a backend admin panel for extended functionality. (Developer, n.d.)

3.2 Process Flow.

The **Process Flow Diagram** illustrates the sequential interactions between users, scholars, and the admin system, outlining how key actions—such as hiring, reporting, verification, and community engagement—flow through the Imaan Link platform in real time.

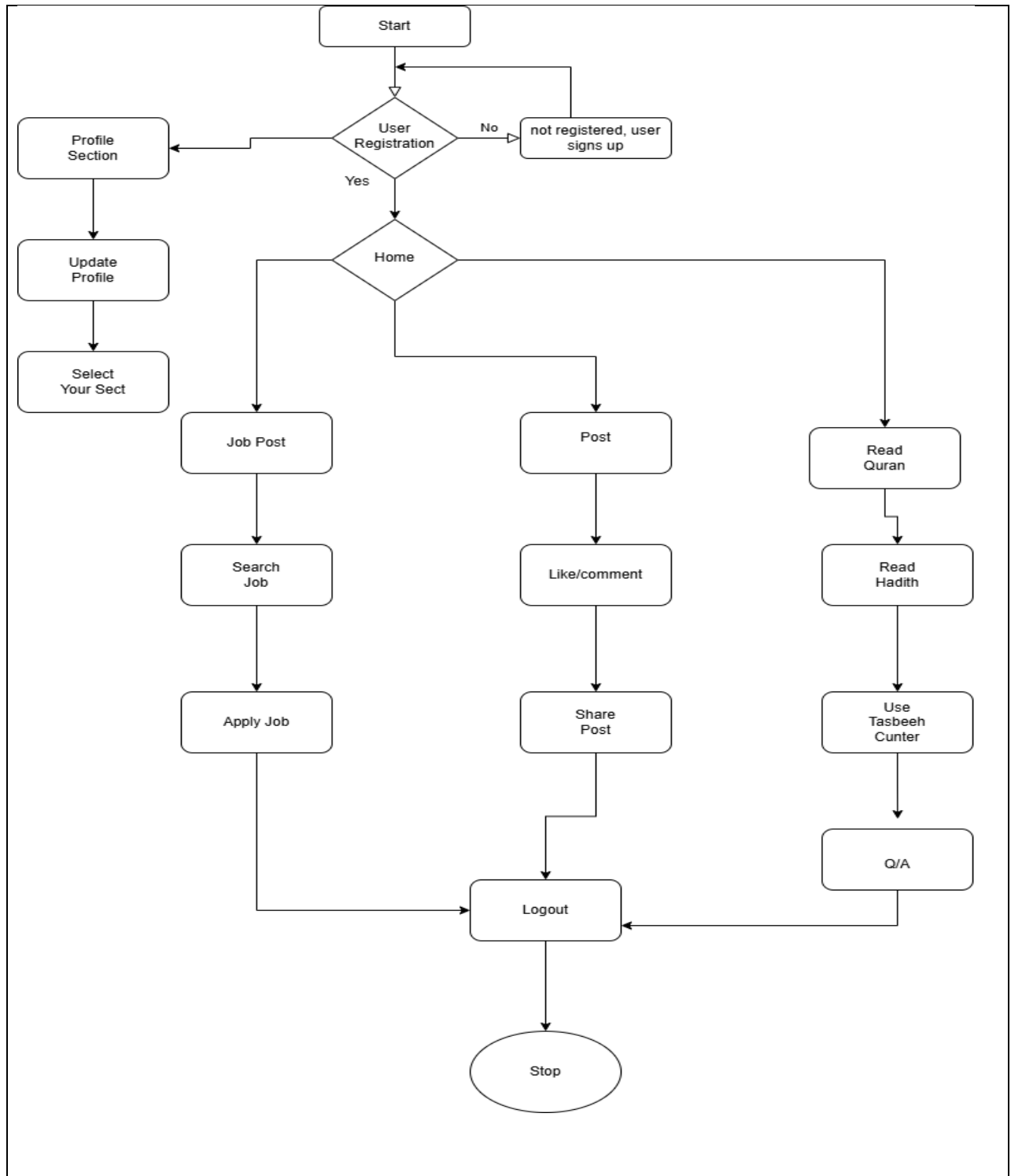


Figure 3:Process Flow

3.3 Process Flow/Representation

This flowchart provides a detailed overview of the interactions within the system, starting from user registration to the termination of the session. It highlights decision points like email and account verification, ensuring secure access for users. Additionally, the admin section supports efficient management of system operations, enhancing overall functionality and user experience. The workflow ensures streamlined navigation for both users and administrators.

3.3.1 Flow Description

- **User Login / Registration**
 - The process starts with the user accessing the app and logging in or registering using secure Firebase Authentication.
- **Dashboard Access**
 - After successful login, the user is directed to a personalized dashboard, showing Posts, upcoming events, and recommendations.
- **Browse Jobs**
 - The user can explore jobs area to create and apply for jobs
- **Notification Triggered**
 - The users receive a real-time notification about the request via Firebase Cloud Messaging (FCM).
 - The scholar accepts, declines, or reschedules the request. Status is updated for the user immediately.
- **Event Creation & Participation**
 - Users or mosque admins can create events. Other users can view details, and receive reminders in community section
- **Resource Access & Learning**
 - Users can access Islamic resources from the digital library (PDFs and Duas).
- **Community Interaction**
 - Users can join discussions, ask religious questions, or respond to posts in a moderated forum.
- **Logout / Exit**
 - Users can log out securely, ensuring session termination and privacy.to include design models that provide a clear and comprehensive understanding of the system's structure and functionality.

3.4 Sequence Diagram

The sequence diagram for Imaan Link shows the steps a user takes when using a platform-based app, accessing hiring procedure, or receiving chats from other users. This is the following Sequence Diagram of our project.

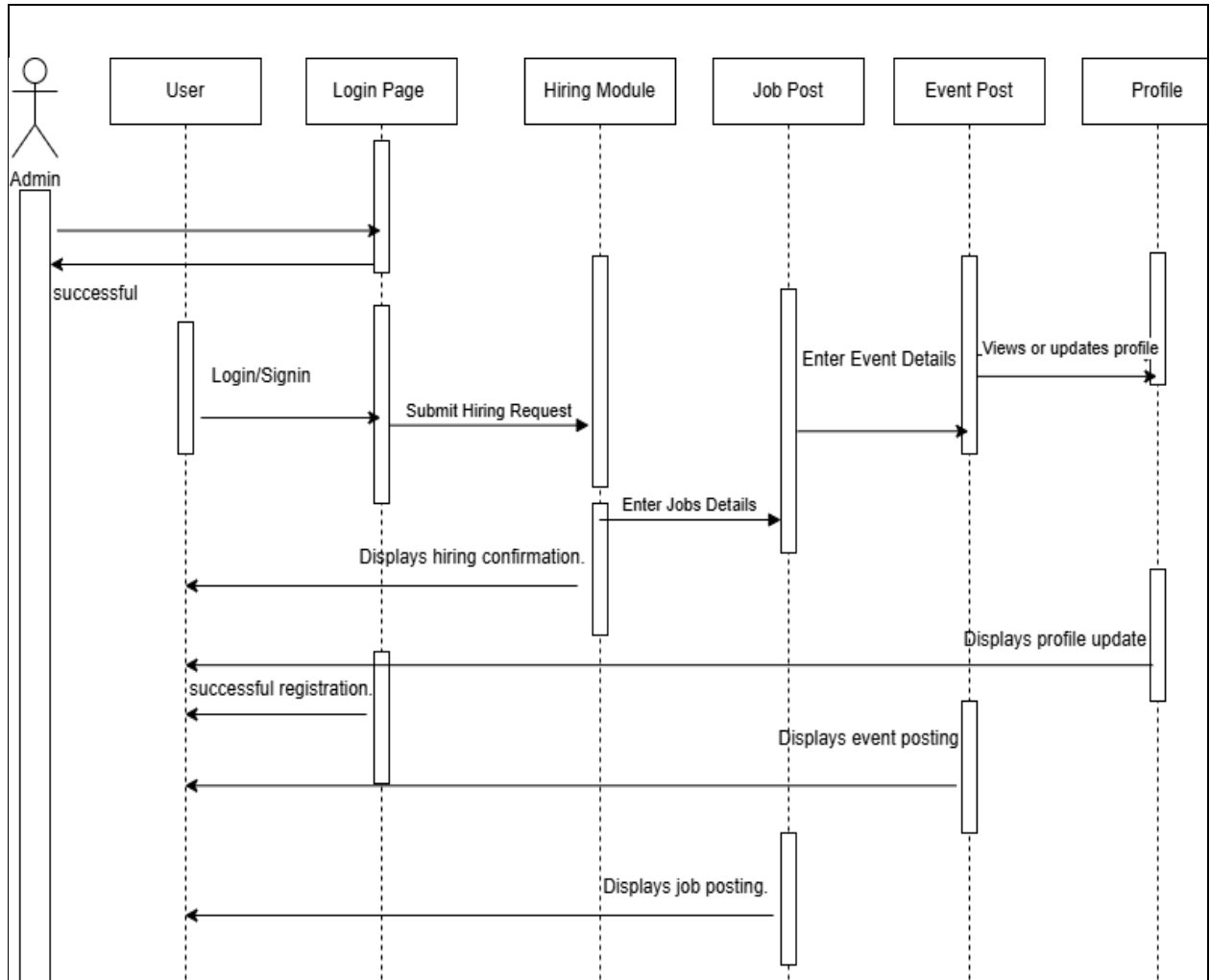


Figure 4:Sequence Diagram

3.5 Class Diagram

The **Class Diagram** represents the structure of the Imaan Link system by defining its core classes—such as User, Scholar, Post, Report, and Certificate—along with their attributes, methods, and relationships.

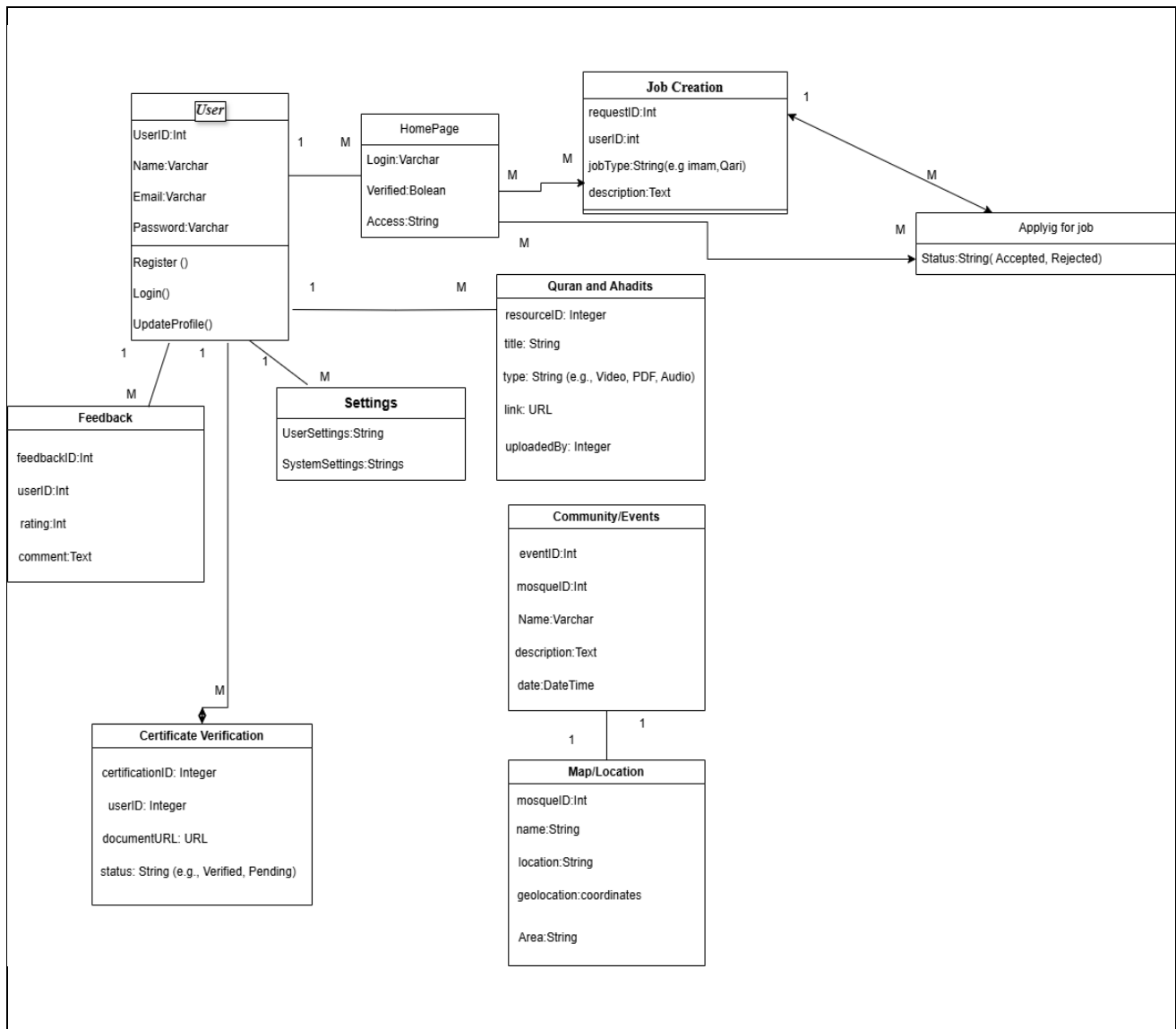


Figure 5:Class Diagram

Chapter 4

Implementation

4 Implementation

The **implementation** of Imaan Link is based on Flutter for cross-platform development and Firebase for backend services, including authentication, database, storage, and notifications. The system follows a modular approach, enabling real-time interactions, secure role-based access, and seamless integration between the Main App and Admin App.

4.1 External APIs

Imaan Link integrates external APIs such as Firebase Authentication, Fire store, Cloud Storage, and Firebase Cloud Messaging to manage secure login, real-time data, media handling, and push notifications.

Table 13: Describes the API'S and their description

API Name	Description	Purpose
Firebase Authentication	Secure login system	Handles authentication
Firebase	Real-time DB	Stores all user/scholar data
Firebase Cloud Messaging	Notification System	Sends alerts & updates

4.2 User Interface

- **Main Dashboard:** Personalized feed (events, Job tab).
- **Tabs:** Home, Scholars, Community, Quran Tasbeeh etc.
- **Language Toggle:** English (Arabic and Urdu planned).
- **UI Design:** Inspired by Islamic minimalism with green and white tones. (Ibrand, n.d.)

4.2.1 Dashboard and Job Tab

The dashboard serves as the central hub for users, displaying key information such as posts, Job tab, and More Options. It offers a clean and intuitive layout for easy navigation. It ensures that users have quick access to essential features.



Figure 6:Dashboard

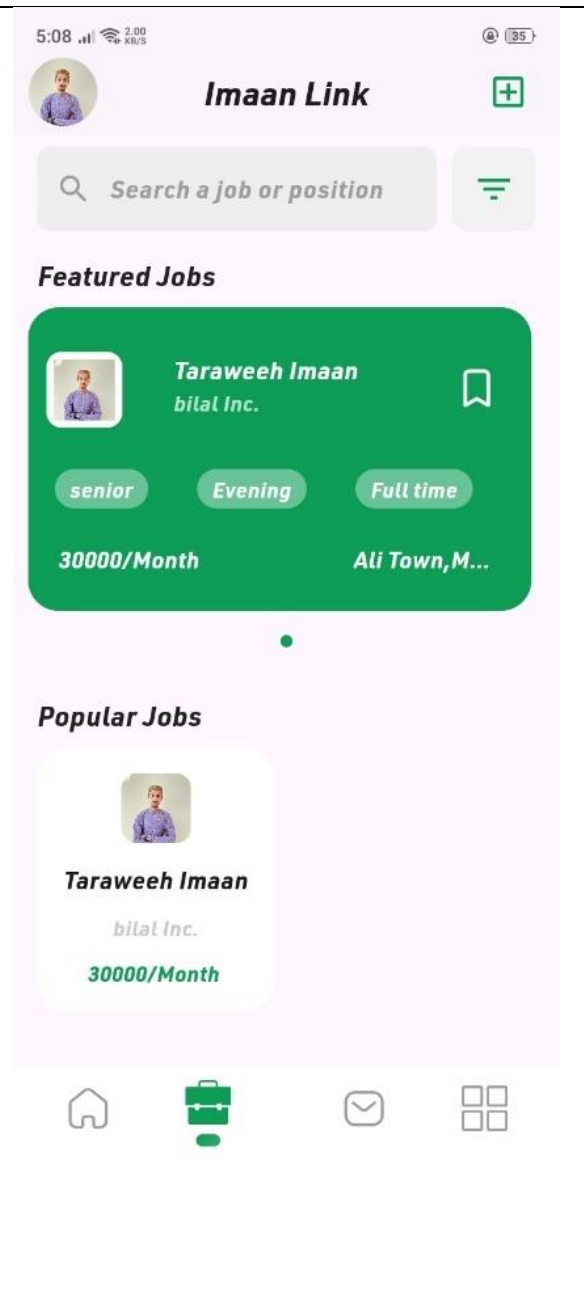


Figure 7:Job Tab

4.2.2 Profile and More Options

These screens show about user profile and more options where users can access Quran, Community Duas etc.

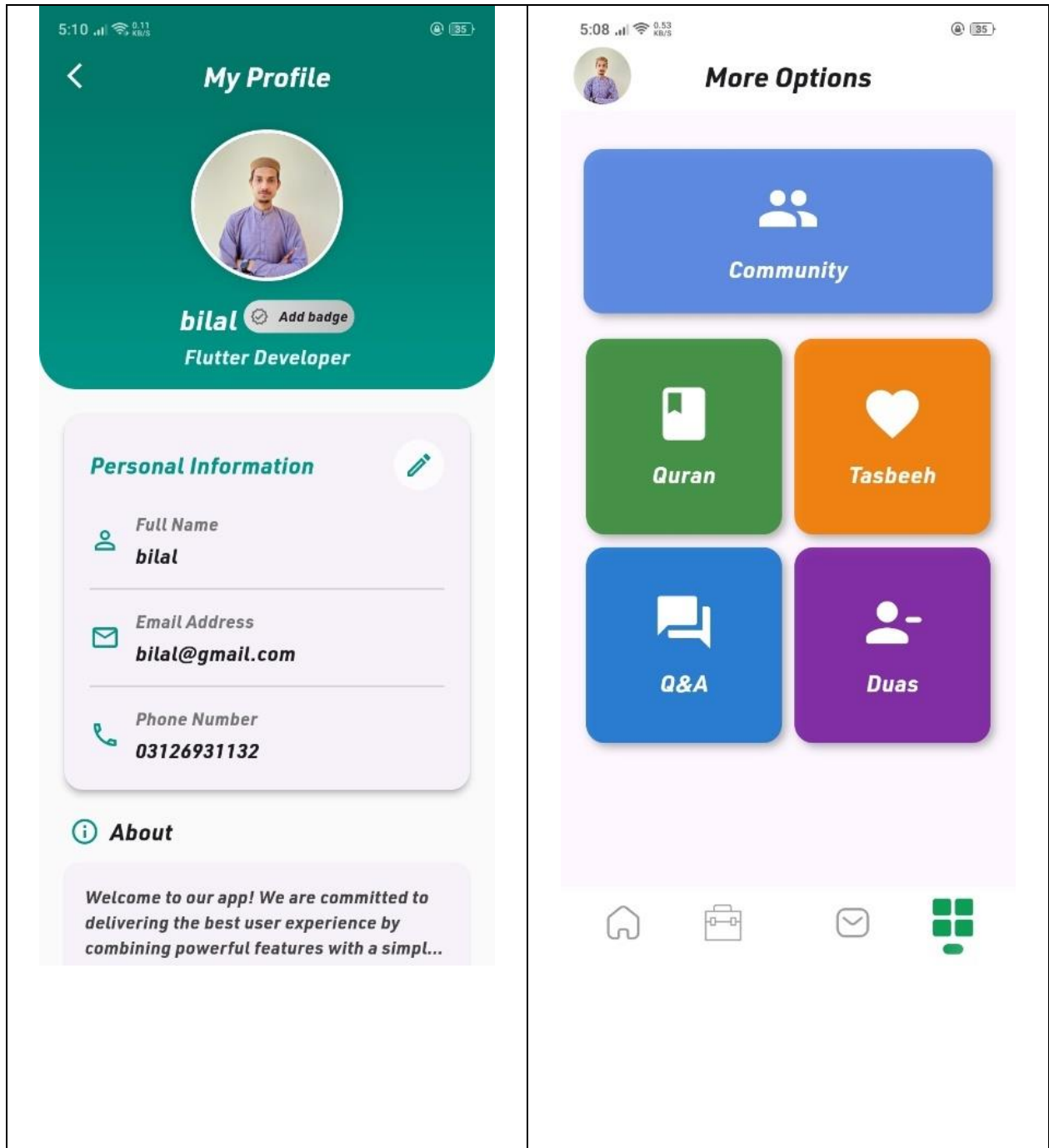


Figure 9: User Profile

Figure 8: More Options

4.2.3 Admin Dashboard and management

The admin dashboard screen instantly upon landing shows how no of users no of reports and user analysis while admin management screen helps to manage admins (update delete and add admin)

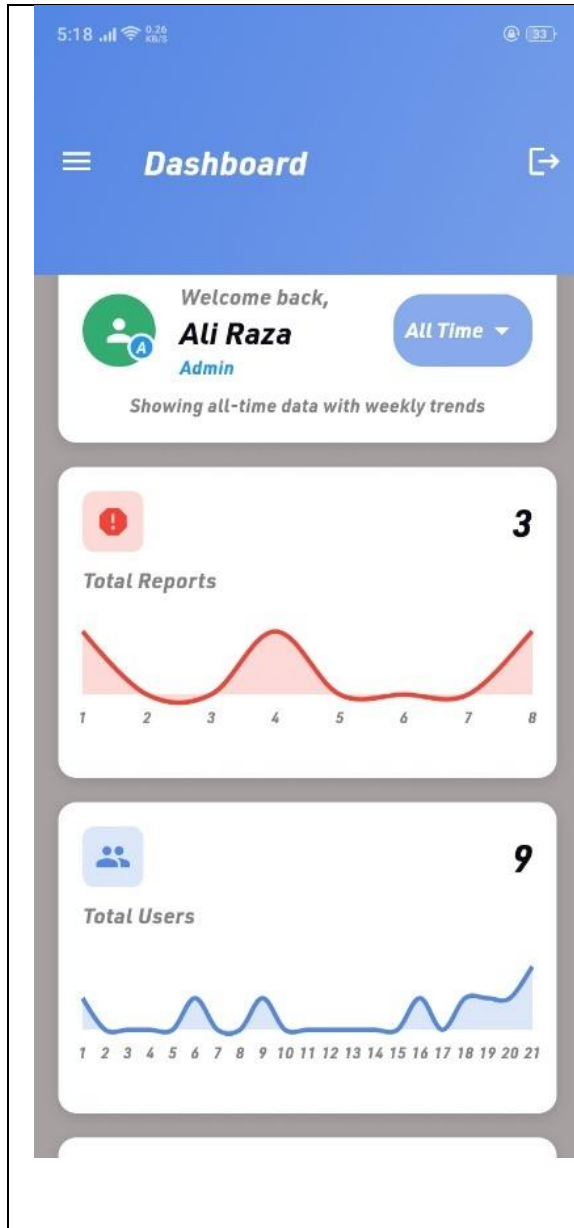


Figure 11:Admin Dashboard

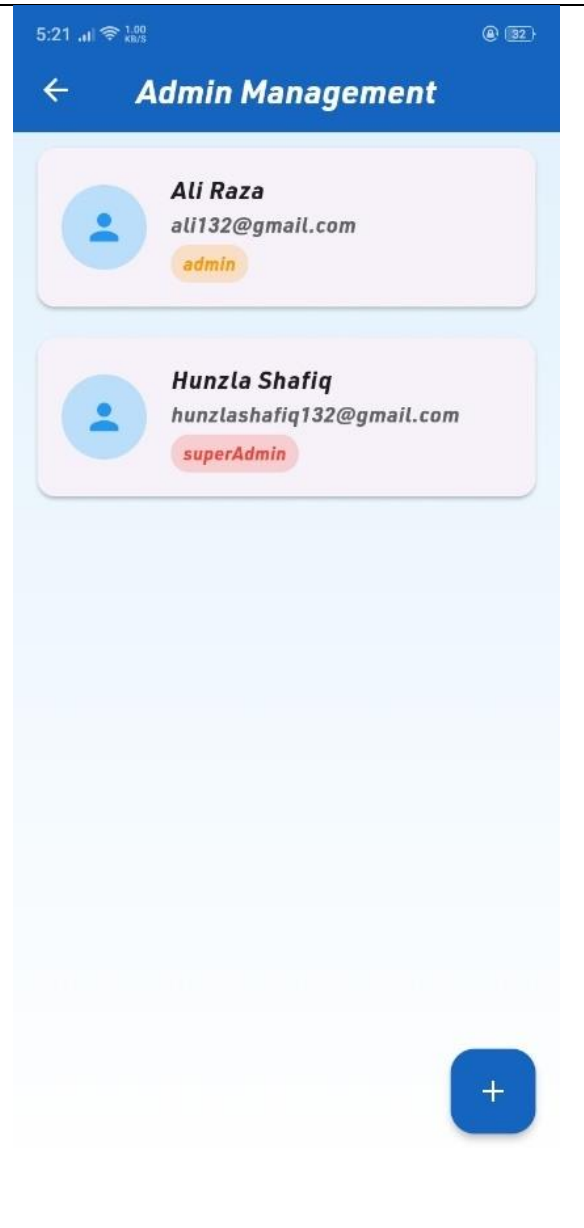


Figure 10:Admin Management

4.2.4 Verification request and Report History

This screen showcases the verification screen where users asked to get verified after uploading their certificate it also shows users whole profile for easy access to fetch all information in one go, while report history screen shows how many reports were solved dismissed and people warned.

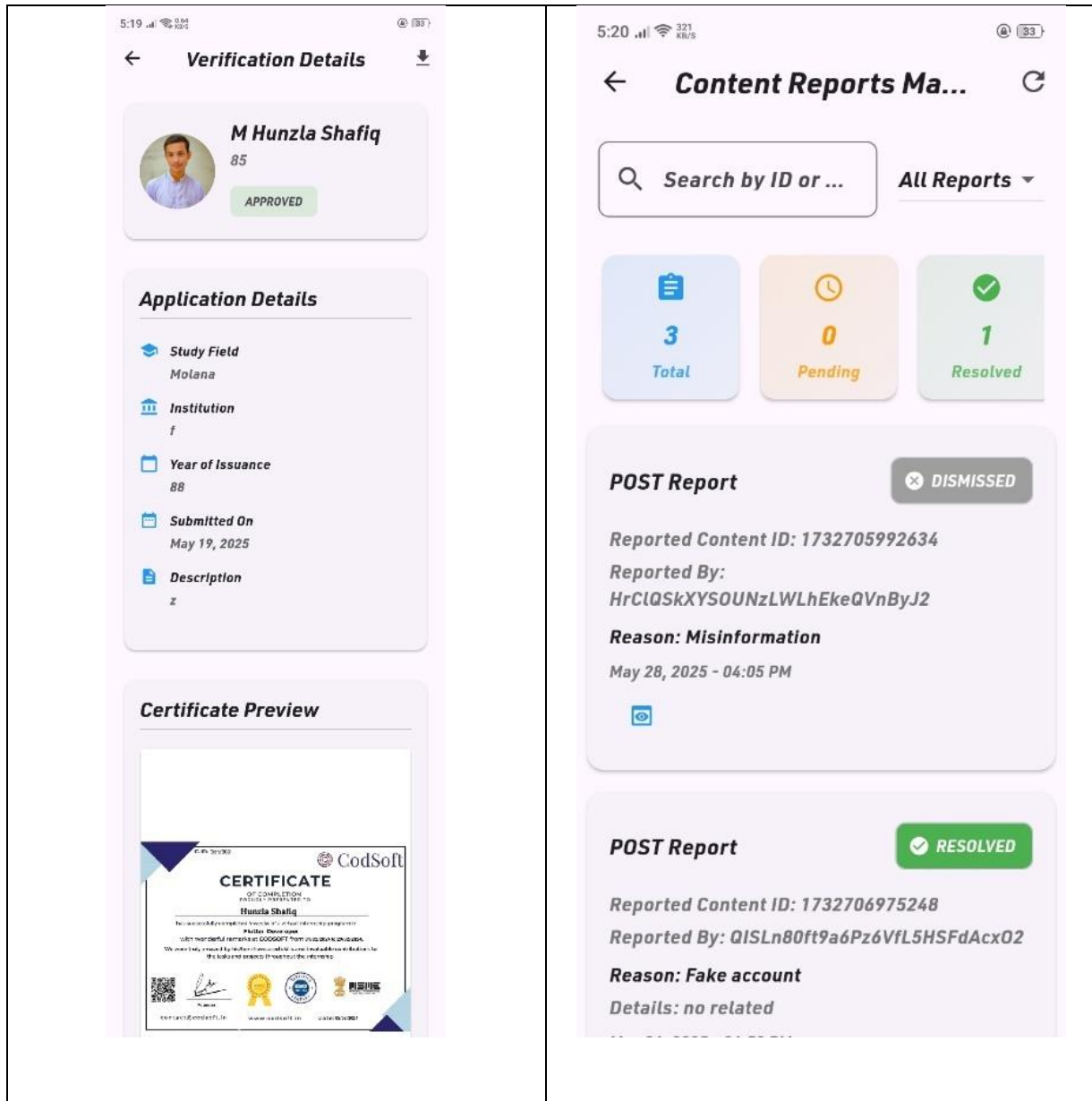


Figure 13:Verification

Figure 12:Report History

4.2.5 User management and report content view

This screen allows admins to see and handle no users (admin can delete users) and the post content screen shows when someone reported about a post (any kind of video and audio) it shows the whole content if its volatile admin will delete post if not it will be ignored and marked as solved.

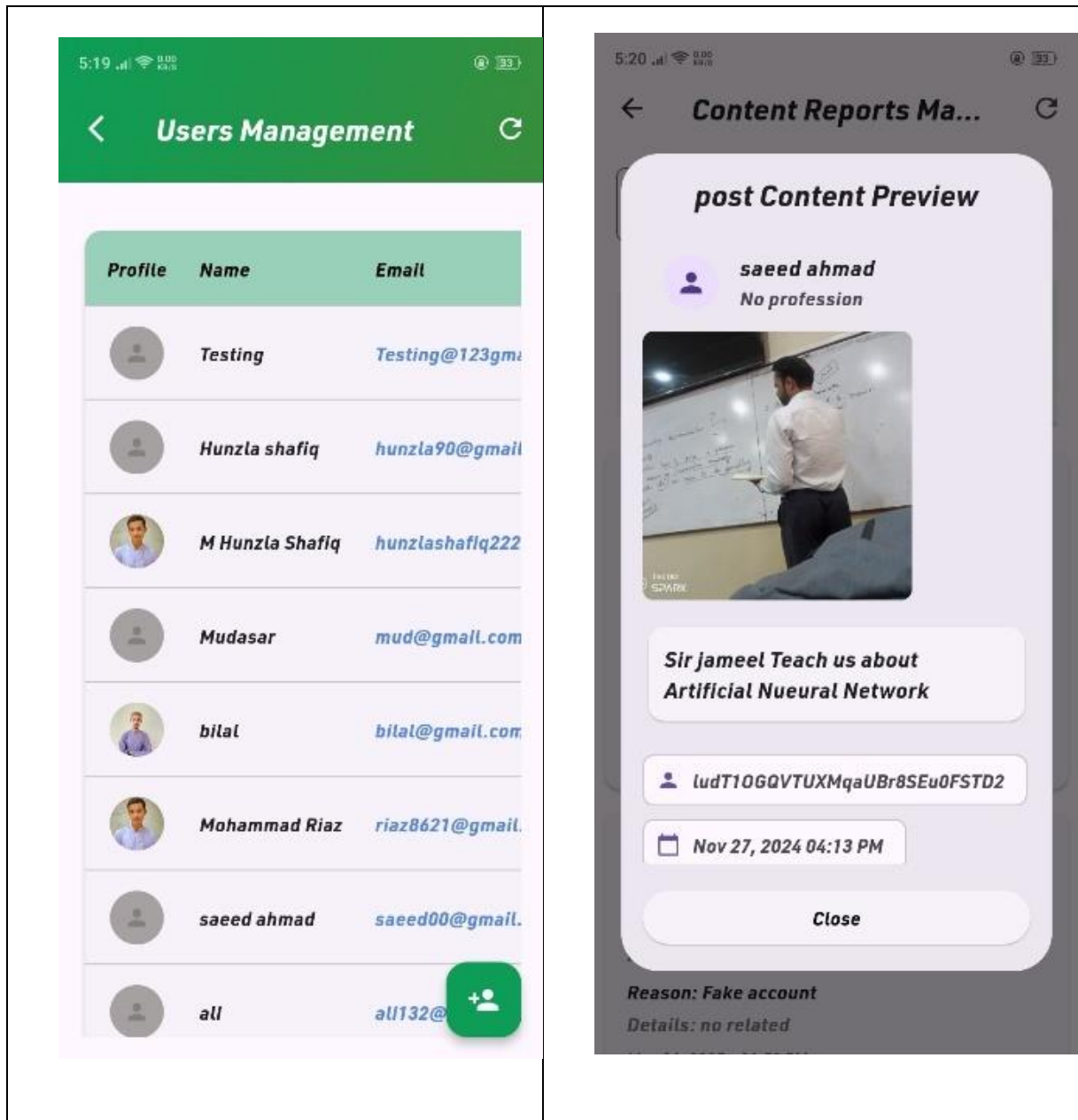


Figure 15:User Management

Figure 14:Post Review

4.2.6 Verification Requests and New admin Creation

In this screen, the admin can see no of people who applied for verification as a scholar and admin can proceed through further from this point to check the user's certificate while new admin creation screens show how a super admin can create a multiple admins and give them privileges of admins/mods.

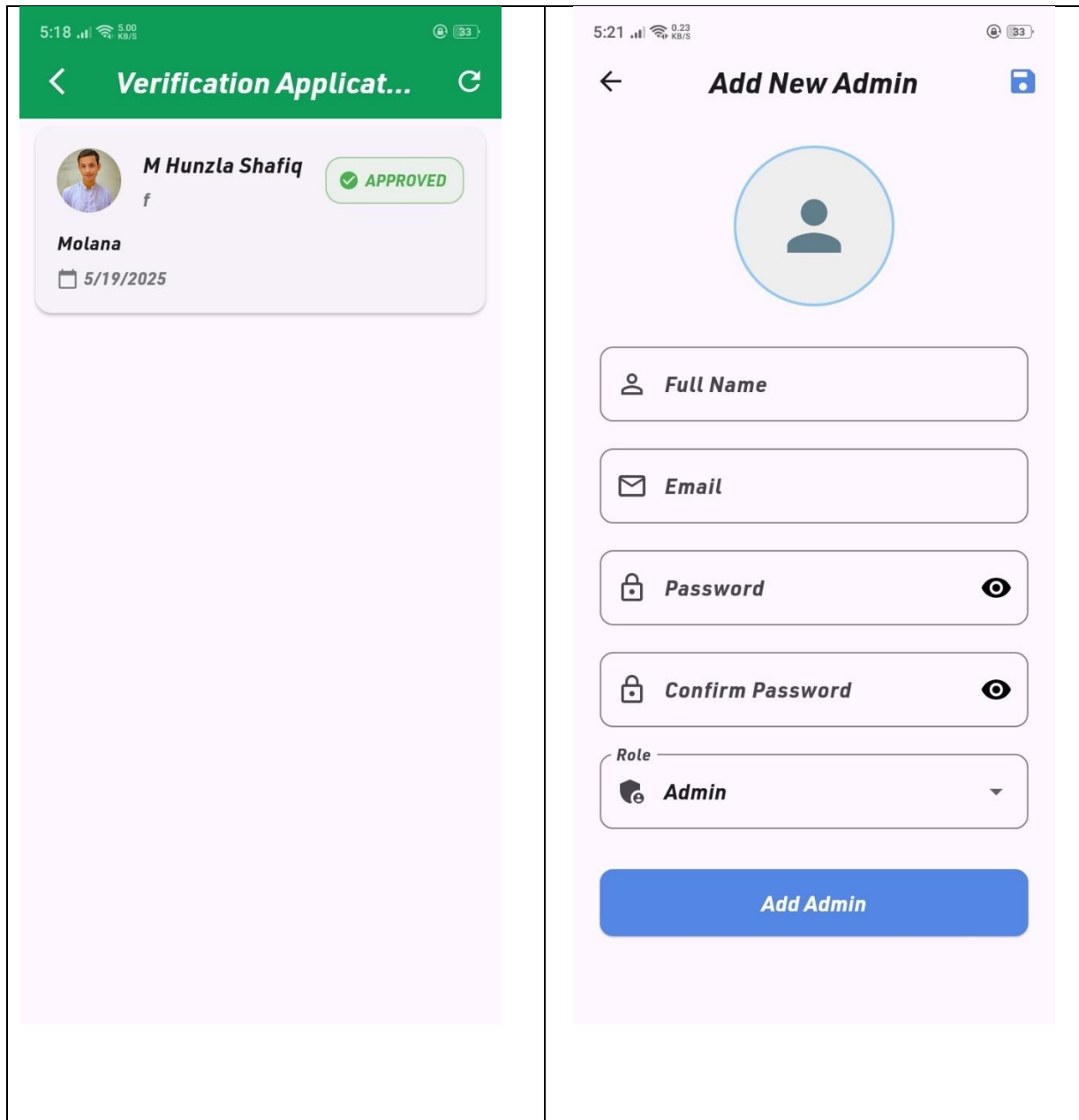


Figure 17:Verification Request

Figure 16:New admin Creation

Chapter 5

Testing and Evaluation

5 Testing and Evaluation

The **testing and evaluation** of Imaan Link involved multiple stages to ensure the reliability, security, and usability of both the Main App and Admin App. Unit testing was conducted to validate individual components such as the tasbeeh counter logic, user authentication, and report submission forms. Integration testing ensured smooth interactions between modules like hiring, verification, and notification delivery. Security was enforced through strict Firebase rules and role-based access control, preventing unauthorized data access. Usability testing was performed with real users who provided feedback on interface design and navigation, leading to several UI improvements. Overall, the system was evaluated to be stable, secure, and user-friendly, meeting the functional and non-functional requirements outlined during the development phase.

5.1 Manual Testing.

Testing across all core features including Manual testing involved step-by-step execution of user actions to identify any functional issues, design inconsistencies, or usability problems. Each screen and feature of the application was manually tested on real Android devices to ensure correct behavior under normal and edge-case scenarios.

- Login/logout
- Hiring process
- Community participation
- Notifications
- Profile editing

5.2 System Testing

Once the system has been successfully developed, testing has to be performed to ensure that the system working as intended. This is also to check that the system meets the requirements stated earlier. Besides that, system testing will help in finding the errors that may be hidden from the users.

Table 14:The below table described the features and expected outcomes of the testing

Feature	Expected Outcome	Status
Login	Redirect to dashboard	Pass
Hire Scholar	Request sent and visible	Pass
Community	Participation	Pass
Real Time chat	Chat completed	Pass

5.3 Unit Testing

Unit testing in the Imaan Link project focused on validating the functionality of individual modules and components to ensure they performed as expected in isolation. Key areas tested included user authentication, where inputs such as email, password, and login status were verified for correct handling; the tasbeeh counter logic, which was tested for goal tracking and reset behavior; and the post creation feature, ensuring that text, image, and video inputs were properly validated and saved. Additional unit tests were applied to the admin functionalities, such as issuing warnings and resolving reports, to confirm the logic executed accurately based on conditions like report counts. These tests were implemented using Flutter's built-in testing framework and helped catch errors early in development, improving code stability and reliability.

Table 15:The below table described the Unit testing using inputs to reach the expected results

Test Case	Input	Expected Output	Result
User Login	Valid creds	Home page	Pass
Create Post	Post Uploaded	Added to DB	Pass
Dua + Quran	Accessed	Accessed	Pass

Unit Testing 1: Login as FYP Committee

Testing Objective: To ensure the login form is working correctly

No.	Test case/Test script	Attribute and value	Expected result	Result
1.	Verify user login after click on the „Login“ button on login form with correct input data	Username: Testing123@g mail.com Password: 123456	Successfully log into the main page of the system as FYP Committee member.	Pass
2.				

Unit Testing 2: Edit Profile

Testing Objective: To ensure the edit profile form is working properly.

No.	Test case/Test script	Attribute and value	Expected result	Result
1.	Click Edit Profile and change profile picture	Upload new profile picture	Profile picture will be changed and new profile picture will be shown in profile and home page	Pass

Unit Testing 3: Creating Job

Testing Objective: To ensure that job is creating and fetching into user profile

No.	Test case/Test script	Attribute and value	Expected result	Result
1	Creating Job	Job is created and fetched	The job will be fetched into user profile	Pass

Unit Testing 4: Applying for job

Testing Objective: To test for users that applying to job is successful.

No.	Test case/Test script	Attribute and value	Expected result	Result
1	Apply for Job	Applied to job and	Applicants' forms should be showed to job maker	Pass

5.4 Functional Testing

Functional testing in the Imaan Link project was carried out to ensure that all system features performed in accordance with the defined functional requirements. This testing validated critical user interactions such as registration, login, hiring a scholar, joining a community, posting content, and submitting feedback. For the Admin App, functional tests confirmed the correct execution of tasks like reviewing reports, issuing warnings, verifying certificates, and updating admin roles. Each function was tested using real user scenarios to ensure that the output matched the expected results, such as displaying the “Verified” badge after approval or triggering real-time notifications when reports were submitted. The functional testing phase played a key role in confirming that every module was operational and delivered the intended user experience without errors.

Functional Testing 1: Login with different roles

Objective: To ensure that the correct page with the correct navigation bar is loaded.

No.	Test case/Test script	Attribute and value	Expected result	Result
1.	Login as a „FYP Committee“ member.	Username: Testing123@gmail.com Password: 123456	Main page for the FYP Committee member is loaded with the FYP Committee navigation bar	Pass
2.				

Functional Testing 2: Normal Posting

Objective: To ensure that the simple image/text post is being posted

No.	Test case/Test script	Attribute and value	Expected result	Result
1.	Make a new post	New post shows at top and posted	The post can be deleted updated ad can be liked	Pass

2.				
----	--	--	--	--

5.5 Integration Testing

Integration testing in the Imaan Link project focused on verifying the interaction between different modules and ensuring that data flowed smoothly across the Main App and Admin App. Key scenarios tested included the hiring process, where a user's request successfully triggered a notification to a scholar and updated the database in real time, and the report system, where a flagged community post was routed to the admin dashboard for moderation. The integration between Firebase Authentication, Firestore, and Cloud Messaging was rigorously tested to ensure that actions such as certificate submissions, post creation, and admin responses were correctly linked across all relevant components. This testing ensured that the modular architecture worked cohesively, delivering a seamless and reliable experience to both users and administrators.

Table 16:The below table shows that the integration testing and its expected results.

No.	Test case/Test script	Attribute and value	Expected result	Result
1.	Login as “FYP Committee” member	Username: Testing123@gmail.com Password: 123	Login successful and the FYP Committee page with its navigation bar is loaded and in the view profile page	Pass
2.	Create Job	Job description and duration of job expiry	Job Created Successfully	Pass
3.	Create Social Post	Choose image or text	Post Uploaded Successfully	Pass

5.6 Automated Testing

Automated testing in the Imaan Link project was implemented using Flutter's built-in testing tools to ensure consistent and repeatable validation of core functionalities across both the Main App and Admin App. Automated test scripts were created to simulate user interactions such as login, navigation between modules, post creation, and tasbeeh goal tracking. These tests helped identify regressions quickly during development and ensured that new features did not break existing functionality. Additionally, Firebase Test Lab was used to perform automated UI testing on multiple devices, verifying layout responsiveness and performance under different screen sizes and operating systems. This automated approach significantly reduced manual testing time and enhanced overall software quality by catching issues early in the deployment cycle.

Tools that are given Below:

Table 17: Describes the Tools and their uses in the app

Tool Name	Tool Description	Applied on [list of related tests cases / FR / NFR]	Results
Firebase	Real time database	Chat saved	pass
Authenticate Login	Real time Authentication	Access Granted	Pass
Flutter	UI	Create user interface	Pass
Dart	Programming Language	Backend Implementation	Pass

Chapter 6

Conclusion and Future Work

6 Conclusion and Future Work

In conclusion, **Imaan Link** successfully provides a modern, secure, and scalable solution for verified scholar engagement, community interaction, and administrative moderation within the Islamic digital space. The dual-app architecture ensures clear separation of user and admin responsibilities, promoting safety and trust. The system meets its objectives through modular design, real-time updates, and role-based access control.

For future work, we plan to introduce **live scholar sessions**, **AI-based content recommendations**, and **full multilingual support**. These enhancements will further expand the platform's accessibility, personalization, and educational reach for the global Muslim community.

6.1 Conclusion

The development of **Imaan Link** marks a significant milestone in the digitization of Islamic religious services. By identifying critical shortcomings in existing systems—such as the lack of verified scholar access, absence of administrative oversight, and fragmented user experiences—this project addresses a real and growing need within the Muslim community. The platform's core innovation lies in its ability to **combine service provision, community engagement, and platform governance** into a single, cohesive system.

Unlike most Islamic mobile applications that focus on either Quranic content, prayer times, or basic tasbeeh counters, Imaan Link was built to solve service-level problems using modern software engineering practices. The introduction of a fully separate Admin App further elevates the project's scope and technical maturity, enabling administrators to moderate community posts, verify scholars through uploaded documentation, track platform analytics, and manage system users—all in real-time. The main user application also goes beyond traditional boundaries by incorporating features such as a goal-oriented tasbeeh counter with Dikr suggestions, a 6-digit code-based private community system, and an integrated verification badge system for scholars. This combination of features not only makes the application technically robust but also ensures high levels of trust, transparency, and spiritual productivity for its users. Technically, the platform demonstrates deep understanding and application of:

Cross-platform development using Flutter

- Firebase integration for authentication, real-time data, cloud storage, and push notifications
- Role-based access control and secure data handling
- Modular design and clean separation of concerns

The application is also highly scalable and maintainable due to its modular architecture, where features are treated as independent services. This enables future extensions and integrations without disrupting the existing system.

- From a social impact perspective, Imaan Link contributes meaningfully by:
 - Digitally connecting communities with qualified religious leaders
 - Enabling safe, moderated religious discourse online
 - Promoting trust through transparent verification systems

Overall, this project stands as a complete solution, both in terms of its technical implementation and its relevance to real-world challenges. It not only meets but exceeds the criteria expected of a Final Year Project, both in complexity and impact.

6.2 Future Work

While Imaan Link delivers a robust and user-ready experience, there are several exciting features and improvements planned for future iterations. These features are intended to further enhance user engagement, widen platform reach, and improve the system's ability to support diverse user needs.

- **Planned Feature Enhancements**
 - Live Q&A and Video Lectures with Scholars
 - A live interaction system where verified scholars can conduct sessions, answer community questions in real-time, and offer lectures through integrated video streaming.
- **AI-Powered Content Recommendations**
 - An intelligent suggestion system that learns from user behavior (e.g., topics liked, events attended, Dikr patterns) to recommend scholars, community posts, and tasbeeh challenges.
- **Multilingual Support for Arabic and Urdu**
 - Full language localization including right-to-left (RTL) support, enabling access to non-English-speaking users and enhancing cultural inclusivity across Muslim regions.
- **Scholar Certification Export Feature**
 - Allowing scholars to download a verified certificate PDF or badge they can use externally, helping build trust outside the app as well.
- **Event-Based Notification System**
 - More advanced notifications including scholar event reminders, upcoming community discussions, or tasbeeh goal milestones.
- **Community Tagging and Search Filters**
 - Advanced post filtering and topic tagging, allowing users to sort by category such as Fiqh, Tafsir, Youth, Women, etc.
- **Scholar Rating System Based on Knowledge Domains**
 - Community-driven ratings to reflect specific knowledge areas (e.g., Fiqh, Hadith, Quranic Sciences) which helps users hire the right scholar for their needs.
- **Integration with Islamic Calendar**
 - Enable scheduling and reminders based on Islamic dates (e.g., Ramadan, Hajj season, Ashura), and context-aware content delivery.

Chapter 7

References

7 References

360, I. (n.d.). *Islam 360 Landing /Home page*. Retrieved from islam 360: <https://theislam360.com/>

Developer, G. F. (n.d.). *Flutter Help*. Retrieved from Flutter for android: <https://docs.flutter.dev/>

Ibrand. (n.d.). *Ibrand landing page Help*. Retrieved from Ibrand: <https://ibrandstudio.com/articles/user-friendly-mobile-app-design-tips-best-practices>

